

Document Knowledge Agent

A smart system that lets you upload documents and ask questions about their content using Retrieval-Augmented Generation (RAG). Instead of scrolling through pages looking for a specific detail, you upload the file once and simply ask.

Under the hood, every document is parsed, split into meaningful fragments, and turned into searchable embeddings. When you ask a question, the system retrieves the most relevant fragments and uses an AI model to generate a grounded answer — with the original source passages attached, so you can always verify where an answer came from.

The goal is simple: turn static documents into a knowledge base you can actually have a conversation with.

[image] [image] [image] [image]

❖ Features

- 📄 **PDF upload** — add new documents to your knowledge base in a click.
- **Content extraction & processing** — text is automatically extracted and cleaned from each document.
- ✂️ **Smart chunking** — documents are split into meaningful fragments for accurate retrieval.
- 🔍 **Semantic search** — finds relevant content by meaning, not just keyword matching.
- 💬 **AI-powered Q&A** — ask questions in plain language and get grounded, source-backed answers.
- 📖 **Source transparency** — every answer links back to the exact

passages it came from.

- ⚡ **Asynchronous document processing** — large documents are handled in the background without blocking the user.
 - ☁ **Cloud-ready architecture** — built to run locally or deploy to the cloud with no code changes.
-



Architecture

```
User
  ↓
Frontend
  ↓
API Gateway
  ↓
Backend Services
  ↓
Document Processing Pipeline
  ↓
Vector Database + AI Models
```

- **Frontend** — where users upload documents and ask questions.
- **API Gateway** — the single entry point for all requests; validates input and routes it to the right service.
- **Backend Services** — orchestrates the RAG pipeline and exposes the core API.
- **Document Processing Pipeline** — extracts, chunks, and embeds document content.
- **Vector Database + AI Models** — stores embeddings for semantic search and generates answers using foundation models.

Each layer has a single, well-defined responsibility, which keeps the system easy to reason about and easy to extend — for example, swapping the AI provider only touches one layer, never the rest of the pipeline.

Tech Stack

Backend

- Python
- FastAPI
- Node.js + Express (API Gateway)

AI / RAG

- Embeddings
- Vector Database
- Foundation Models

Cloud

- AWS Lambda
- API Gateway
- CloudFront
- S3
- Terraform

Frontend

- React
 - TypeScript
-



Engineering Highlights

- **Serverless-first architecture** to minimize idle cost and scale automatically with demand.
- **Clear separation** between the lightweight API Gateway and the heavier document-processing pipeline, so each can scale and evolve independently.
- **Asynchronous ingestion** for larger documents, so processing time never blocks the user experience.
- **Infrastructure as Code** — the entire cloud environment is defined

and reproducible through Terraform.

- **Provider-agnostic design** — the AI/embedding/vector-store providers are fully decoupled from the application logic, so switching providers is a configuration change, not a rewrite.
-



Local Development

Requirements: Docker & Docker Compose.

```
git clone <this-repository>
cd document-knowledge-agent
docker compose up
```

Then open the app at <http://localhost:5173>. Everything — frontend, API, document processing, and AI models — runs locally, with no cloud account required.



Project Status

This is an actively evolving personal portfolio project. The system supports both **local** and **cloud-based** providers for embeddings and language models, selectable through configuration — allowing it to run fully offline for development, or on a real serverless cloud deployment for production use.